

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Best Bike Paths

Acceptance Testing Document

Lorenzo Fiamingo
Alexander Gabriel Fox
Foad Zahedi Labaf

Academic year 2025/2026

Contents

Contents iii

List of Figures v

List of Tables vii

1 Introduction 1

1.1 Purpose 1

1.2 Scope 1

1.3 Version History 1

1.4 Document Structure 1

2 Project Identification 3

2.1 Authors 3

2.2 Repository 3

2.3 Reference Documents 3

3 Installation Setup **5**

3.1 Installation Procedure 5

4 Acceptance Testing **7**

4.1 Test Environment & Preconditions 7

4.2 Scope of Testing 7

4.3 Acceptance Test Cases 8

4.4 Test Coverage 14

5 Quality Assessment **17**

5.1 Implementation Issues 17

5.1.1 Core Issues 17

5.1.2 UX Issues 18

6 Conclusions **19**

7 Effort Spent **21**

8 References **23**

List of Figures

4.1 Trip Origin. 12
4.2 Trip Destination. 12
4.3 Search Origin. 12
4.4 Search Destination. 12
4.5 Search Result. 12

List of Tables

4.1 Test Coverage on Requirements. 14
7.1 The number of hours each group member has worked for this document. . . 21

1 Introduction

1.1 Purpose

The purpose of this document is to verify the acceptability of the other group's implementation of the BBP project through a series of acceptance tests. In particular, this document describes the acceptance testing activities performed on the project and reports the results of each test, with the aim of identifying potential bugs in the code and any inconsistencies with respect to the project description provided in the Requirements Analysis and Specification Document, the Design Document, and the Implementation and Testing Document.

1.2 Scope

The testing activities primarily focus on verifying the interaction between the frontend and backend services of the project (specifically the storage and retrieval of data from the database, authentication mechanisms, internal business logic, and the use of external libraries) in relation to the requirements implemented according to the *ITDv1* document. The tests are performed on the implemented application functions after following the installation instructions outlined in the *ITDv1* document available in the corresponding GitHub repository. These testing activities do not include frontend user interface testing; however, some observations regarding potential interface improvements are included within the relevant test cases.

1.3 Version History

- Version 1: 08/02/2026

1.4 Document Structure

1. **Introduction**: A brief overview of the contents of the document and its purpose.
2. **Project Identification**: This section identifies the software project under analysis.
3. **Installation Setup**: It describes the steps taken to install the prototype.

4. **Acceptance Testing**: The executed test cases derived from the RASD requirements and the implemented features.
5. **Quality Assessment**: An evaluative section focusing on the overall quality of the project.
7. **Conclusions**: Summarizes testing outcomes and critical defects, including recommendations for essential features missing from the specifications.
8. **Effort spent**: Includes the amount of time dedicated by each member to the realization of each section of the document.
9. **References**: Reference to documents or tools used for writing this document.

2 Project Identification

This section details the acceptance testing and quality assessment performed on the BBP application. The analysis is based on the artifacts and documentation provided by the development team.

2.1 Authors

The project was designed and implemented by:

- **Alice Berta**
- **Francesco Danese**

2.2 Repository

The source code, along with deployment files, is hosted at the following URL:

https://github.com/fradane/DaneseBerta

2.3 Reference Documents

The testing strategy and verification criteria were derived from the following deliverables:

- **Requirements Analysis and Specification Document (RASD)** - Version 2.0
- **Design Document (DD)** - Version 2.0
- **Implementation and Testing Document (ITD)** - Version 1.0

3 | Installation Setup

This section describes the procedure followed to deploy the testing environment. The installation was performed strictly adhering to the instructions provided in **Chapter 6** of the **ITD**.

3.1 Installation Procedure

The system was set up in a hybrid environment to simulate a realistic usage scenario:

- **Server-side**: The backend microservices were deployed using **Docker Compose** on a local machine, ensuring isolation and dependency management as recommended in ITD Section 6.1.

  - After cloning the repository from the corresponding URL and downloading all the required software for running the application's backend, the environment variables required to define the secrets used for generating both access and refresh tokens were configured.
    All service images were then built by executing the command `docker compose build`.
    Finally, the server was started using the command `docker compose up`.
    All updates to the database were verified using the MongoDB Compass app.

- **Client-side**: The mobile application was compiled and installed on two distinct targets:
  1. **iOS Simulator** for rapid functional testing.
  2. **Physical Device** for real-world evaluation.

  - To run the client application, we installed Flutter and configured the API endpoint in `lib/controller/api_config.dart` to point to the server's local IP address (e.g., `http://10.1.1.217:3000`). Once configured, we built and launched the app on the connected device by running the command `flutter run -d <device_id>` from the `src/client/bbp_mobile` directory. To discover the `<device_id>` we ran the commend `flutter devices`.

4 | Acceptance Testing

This chapter details the User Acceptance Testing (UAT) activities performed on the BBP application. The test suite was designed based on the Functional Requirements specified in the **RASD (v2.0)**, while strictly adhering to the implementation scope defined in the **ITD (v1.0)**.

Consequently, requirements explicitly marked as "Not Implemented" in the ITD were excluded from this verification phase. The primary goal was to validate the software against the delivered specifications and ensure robustness in the implemented flows.

4.1 Test Environment & Preconditions

Unless otherwise specified in the individual test cases, the following baseline environment is assumed:

- **Client**: Mobile Application running on an iOS device (physical or simulated) via Flutter engine. The application was used exclusively through its graphical interface, following realistic user interactions.
- **Server**: Backend Microservices (Account, Trip, Path) deployed and running via Docker containers.
- **Connectivity**: Active internet connection for external API access (OpenStreetMap, OpenMeteo).

4.2 Scope of Testing

Testing covered the following functional areas:

- Application startup and overall stability
- Authentication and account management (login, signup, logout, account deletion)
- Session handling and permission management
- Map visualization and path search
- Manual trip creation and validation
- Trip storage and visualization
- Trip publishing and visibility management
- Profile-related features and user settings

- Error handling and crash behavior
- Whenever applicable, both registered and non-registered user flows were tested.

4.3 Acceptance Test Cases

Authentication Module

Sign Up Process & Validation
AT1: Should register a new user with valid data

- **Precondition**: App is installed and user is on the Sign Up screen.
- **Action**: User enters Name, Surname, unique Username, valid and unique Email, valid Password, and clicks Confirm.
- **Expected result**: The account is created successfully. The system initiates the verification process (if applicable) or logs the user in.
- **Requirements verified**: R1, R2, R3
- **Outcome**: *Passed*

AT2: Should enforce a non-empty email field

- **Precondition**: User is on the Login screen.
- **Action**: User leaves the email field empty and enters a password, then presses Login.
- **Expected result**: Login is rejected and an error is shown.
- **Requirements verified**: R1, R2
- **Outcome**: *Passed*

AT3: Should enforce a non-empty password field

- **Precondition**: User is on the Login screen.
- **Action**: User enters a valid email and leaves the password field empty, then presses Login.
- **Expected result**: Login is rejected and an error is shown.
- **Requirements verified**: R1, R3
- **Outcome**: *Passed*

AT4: Should enforce a non-empty name field

- **Precondition**: User is on the Sign up screen.
- **Action**: User attempts sign-up leaving name field empty.
- **Expected result**: Sign-up is rejected and an error is shown.
- **Requirements verified**: R1
- **Outcome**: *Passed*

AT5: Should enforce a non-empty surname field

- **Precondition**: User is on the Sign up screen.
- **Action**: User attempts sign-up leaving surname field empty.
- **Expected result**: Sign-up is rejected and an error is shown.
- **Requirements verified**: R1

- **Outcome**: *Passed*

AT6: Should enforce valid email format
- **Precondition**: User is on the Sign Up screen.
- **Action**: User fills all fields correctly but enters an invalid email format (e.g., "user.name" without domain or "@"). Clicks Confirm.
- **Expected result**: The system displays an error message indicating the email format is invalid and blocks registration.
- **Requirements verified**: R2
- **Outcome**: *Passed*

AT7: Should reject duplicate usernames
- **Precondition**: A user is already registered with a certain username.
- **Action**: Attempt to sign up a new user using the same username.
- **Expected result**: The system displays an error and prevents account creation.
- **Requirements verified**: R4*
- **Outcome**: *Passed*
- **Note**: This requirement is not explicitly traced, but needed for R4.

AT8: Should reject duplicate email addresses
- **Precondition**: A user is already registered with a certain email.
- **Action**: User attempts to sign up a new user using the same email.
- **Expected result**: The system displays an error "Email already registered" and prevents account creation.
- **Requirements verified**: R2
- **Outcome**: *Passed*

AT9: Should validate password complexity
- **Precondition**: User is on the Sign Up screen.
- **Action**: User enters valid details and a password shorter than 8 characters. Clicks Confirm.
- **Expected result**: An error message is displayed and registration is blocked.
- **Requirements verified**: R3
- **Outcome**: *Passed*

Login & Session Management
AT10: Should allow login with username
- **Precondition**: A registered and verified user exists with a known username.
- **Action**: User enters username and correct password. Clicks Login.
- **Expected result**: Login is successful and the user lands on the Home screen.
- **Requirements verified**: R4
- **Outcome**: *Failed*
- **Defect Description**:
    1. **Blocking Issue**: The login is rejected by the server. Although the ITD states that the prototype auto-verifies emails, the database entry defaults

email verification to `false` upon creation, and the Backend enforces verification check on sign-in.

2. **Design Flaw**: There is an ambiguity in credential resolution. If a user registers with a username formatted as an email address (distinct from their actual email), the login query incorrectly targets the email field instead of the username field, causing authentication failure.

- **Recommendation**: Ensure the sign-up handler sets email verification to `true` by default for the prototype, and implement a regex check during sign-up to prevent usernames from having email-like formats.

AT11: Should allow login with email address
- **Precondition**: A registered user exists with a known email.
- **Action**: User enters email address and correct password. Clicks Login.
- **Expected result**: Login is successful, recognizing the email as a valid identifier.
- **Requirements verified**: R4
- **Outcome**: *Failed*
- **Defect Description**: The login is rejected by the server. Although the ITD states that the prototype auto-verifies emails, the database entry defaults email verification to `false` upon creation, and the Backend enforces verification check on sign in.
- **Recommendation**: Ensure the sign-up handler sets email verification to `true` by default for the prototype.

AT12: Should disallow login with invalid credentials
- **Precondition**: User account exists
- **Action**: User enters wrong email or password
- **Expected result**: Error notification is shown
- **Requirements verified**: R4
- **Outcome**: *Passed**
- **Note**: The test cannot properly check if the requirement is satisfied because the error notification is always shown due to the blocking issue described in AT6.

AT13: Should invalidate session on logout
- **Precondition**: User is currently logged in.
- **Action**: User clicks the Log Out button.
- **Expected result**: User is redirected to the Login screen and no longer has a valid token.
- **Requirements verified**: R6
- **Outcome**: *Passed*

User Profile Management

AT14: Should display correct profile information
- **Precondition**: User is logged in.
- **Action**: User navigates to the Profile tab.

- **Expected result**: The displayed Username and Email match the registered account details.
- **Requirements verified**: R7
- **Outcome**: *Passed*

Path Finding Service

AT15: Should allow guest users to search for paths
- **Precondition**: User is not logged in (Guest).
- **Action**: User taps the search path button in the landing page and uses the path finding service.
- **Expected result**: User can use the path finding service.
- **Requirements verified**: R10
- **Outcome**: *Passed*

AT16: Should allow authenticated users to search for paths
- **Precondition**: User is logged in.
- **Action**: User enters in search tab and uses the path finding service.
- **Expected result**: User can use the path finding service.
- **Requirements verified**: R10
- **Outcome**: *Passed*

AT17: Should rank paths by calculated score
- **Precondition**: User selects origin and destination points in an area with at least one possible published route.
- **Action**: User taps the *find best path* button
- **Expected result**: Paths are ordered based on "Score" (combining distance and road status), not solely by shortest distance.
- **Requirements verified**: R11, R12, R27
- **Outcome**: *Failed*
- **Note**: The score calculation described in ITD differs from RDD Assignment specifications.
- **Defect Description**:
  1. **Inconsistent Score Aggregation**: The scoring update mechanism exhibits non-deterministic behavior. In some test runs, the score updates correctly after a new trip is published; in other identical scenarios, the score remains static, failing to incorporate the new feedback.
  2. **Random Data Unavailability**: The appearance of published paths in search results is erratic. Successfully published paths randomly fail to appear in subsequent searches.
- **Observation**: The flakiness of these bugs makes reliable reproduction difficult, as the system behaves correctly in approximately 50% of the attempts. The failure does not appear to be caused by the geographical precision of the search query: as demonstrated by the attached screenshots from our test sessions, using the exact

same coordinates yields empty results in some attempts. This rules out geospatial matching tolerance (100m) as the root cause.

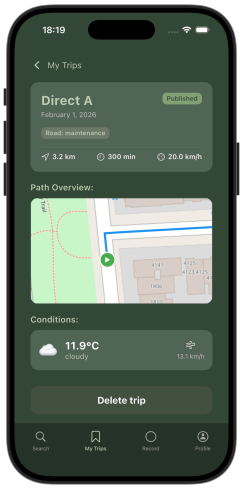


Figure 4.1: Trip Origin

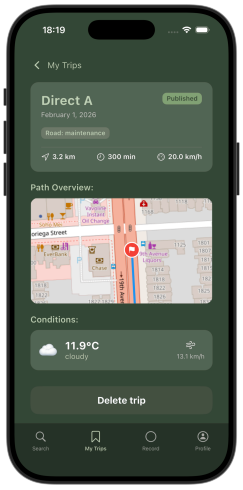


Figure 4.2: Trip Destination



Figure 4.3: Search Origin



Figure 4.4: Search Destination

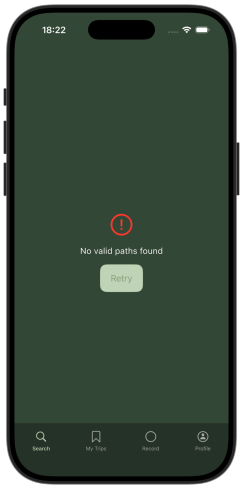


Figure 4.5: Search Result

AT18: Should display distance and score

- **Precondition**: User selects origin and destination points in an area with at least one possible published route.
- **Action**: User taps the *find best path* button
- **Expected result**: The UI shows the total distance and the calculated quality score.
- **Requirements verified**: R13
- **Outcome**: *Passed*
- **Note**: The score calculation described in ITD differs from RDD Assignment specifications.

Trip Recording & Weather

AT19: Should manually create a trip

- **Precondition**: User is authenticated.
- **Action**: User creates a new manual trip specifying travel points and start/end times, average speed, and road status.
- **Expected result**: The trip is saved. The system shows the trip in the "My Trips" section.
- **Requirements verified**: R16, R20
- **Outcome**: *Passed*
- **Note**: The location coordinates are not specified by the user using street names, but by tapping the location on the map.
- **Observation**: While field validation (negative speed, missing fields) is not explicitly traced, it is performed by the app.

AT20: Should save paths with many segments
- **Precondition**: User is inserting a manual trip.
- **Action**: User adds many (usually more than seven) path segments when creating the trip.
- **Expected result**: The trip is saved.
- **Requirements verified**: R16
- **Outcome**: *Failed*
- **Defect Description**: Creating a manual trip with multiple path segments causes the app to show an error.

AT21: Should display user saved trips
- **Precondition**: User has previously saved at least one trip.
- **Action**: User opens My Trips screen.
- **Expected result**: All user saved trips are displayed.
- **Requirements verified**: R21
- **Outcome**: *Passed*

AT22: View trip details
- **Precondition**: User has previously saved at least one trip.
- **Action**: User selects a trip from My Trips
- **Expected result**: Trip details are shown correctly
- **Requirements verified**: R21
- **Outcome**: *Passed*

AT23: Should fetch historical weather data for trips
- **Precondition**: A manual trip is created.
- **Action**: Open the details view of the saved trip from "My Trips".
- **Expected result**: Temperature, wind speed, and weather condition are displayed alongside the trip statistics.
- **Requirements verified**: R21, R33, R34, R35
- **Outcome**: *Passed*

Trip Publishing

AT24: Should publish multiple trips simultaneously
- **Precondition**: User has one or more "Unpublished" trips in their history.
- **Action**: User selects multiple trips (by long-pressing them) and click the "Publish" button.
- **Expected result**: The selected trips' status changes to "Published".
- **Requirements verified**: R26, R27
- **Outcome**: *Passed*

4.4 Test Coverage

Requirement	Verifying Tests
R1	AT1, AT2, AT3, AT4, AT5
R2	AT1, AT2, AT6, AT8
R3	AT1, AT3, AT9
R4	AT7, AT10, AT11, AT12
R5	*Not Implemented*
R6	AT13
R7	AT14
R8	*Not Implemented*
R9	*Not Implemented*
R10	AT15, AT16
R11	AT17
R12	AT17
R13	AT18
R14	*Not Implemented*
R15	*Not Implemented*
R16	AT19, AT20
R17	*Not Implemented*
R18	*Not Implemented*
R19	*Not Implemented*
R20	AT19
R21	AT21, AT22, AT23
R22	*Not Implemented*

Requirement	Verifying Tests
R23	*Not Implemented*
R24	*Not Implemented*
R25	*Not Implemented*
R26	AT24
R27	AT17, AT24
R28	*Not Implemented*
R29	*Not Implemented*
R30	*Not Implemented*
R31	*Not Implemented*
R32	*Not Implemented*
R33	AT23
R34	AT23
R35	AT23

Table 4.1: Test Coverage on Requirements

5 | Quality Assessment

This section outlines observations regarding the code quality, documentation consistency, and user experience (UX) derived from the testing session.

5.1 Implementation Issues

5.1.1 Core Issues

Login
The most critical issue found is a direct contradiction between the architectural decisions described in the ITD and the actual backend logic.
The ITD states that email verification is skipped ("auto-verified") for this prototype. However, the database initialization logic sets the `emailVerification` flag to `false` by default, and the login handler enforces a check on this flag.
This creates an immediate obstacle for any new user, blocking access right after registration. The code should explicitly mirror the "prototype assumptions" documented in the deliverables.

Authentication Session persistence
Although it is not explicitly specified in the implementation, the application does not preserve the authenticated session after being closed and reopened.
This results in the user to being redirected to the login screen after each application restart.

Path Finding Service
The path finding feature demonstrates a non-deterministic behavior where paths randomly fail to appear in search results or don't incorporate new information when calculating the quality score, suggesting some error in the Backend that needs addressing.

Trip Length
When creating a trip manually, the application fails to store trips containing a large number of segments (typically more than seven) in the database.

5.1.2 UX Issues

While the visual interface is clean, two specific interaction patterns degrade usability:

Hidden Functionality (The "Long Press" Issue)
In the "My Trips" section, publishing a trip requires a long-press gesture on the list item. There is no visual cue, icon, or instruction indicating this possibility. Users rely on standard tap interactions; hiding core functionality behind an invisible gesture makes the feature undiscoverable without external help.

Map Manual Entry Inefficiency
The "Manual Trip" creation flow assumes the user is near the starting point. Lacking a text-based search (Geocoding), the user is forced to manually drag and zoom the map to find a remote location. If a user wants to log a trip taken in a different city, the manual panning process becomes very slow.

6 | Conclusions

Authentication and Account Management
Authentication tests focused on access control and credential handling. Password rules are enforced correctly. Account creation works as expected, and duplicate identifiers are correctly rejected.
Login currently is not working. Authentication session persistence across app restarts is not supported.

Map and Path Search Functionality
Map rendering works correctly. Basic navigation and marker placement are responsive. Path search can be triggered, but sometimes no valid routes are returned in the tested scenarios.

Trip Recording and Management
Trip creation and visualization in the My Trips section work correctly.
Trips are not able to include too many segments. Adding several points often makes the app display an error. Automatic trip recording was not observed since it's not implemented.

Trip Visualization, Publishing, and Weather Data
Saved trips appear correctly in the trip list, and detail views are accessible. Publishing via long-press interaction works. Published trips cannot be reverted to private status. Weather data is shown on trips when services are available.

Limitations of Testing
Testing was manual and focused on functional behavior. Performance, security, and usability were not evaluated.

Final Thoughts
The app has good potential. Fixing the discussed problems would significantly elevate the prototype from "technically functional" to "actually usable".

7 | Effort Spent

	Foad Zahedi Labaf	Lorenzo Fiamingo	Alexander Gabriel Fox
Section 1	0	0	1
Section 2	0	1	0
Section 3	2	2	2
Section 4	11	10	6
Section 5	3	4	3
Section 6	1	1	1

Table 7.1: The number of hours each group member has worked for this document

8 References

- **Typst** language and its online editor were used to build this document.
- **GitHub** was used for version control and to automate diagram generation.
- **Google Docs** was used for collaborative writing.
- **Xcode** and **SwiftUI** were used to create UI mockups.
- **ChatGPT** and **Grammarly** were used to refine grammar, assist with code development, and automate error checking.